

A Lyapunov Drift-Plus-Penalty Method Tailored for Reinforcement Learning with Queue Stability

Wenhan Xu, *Graduate Student Member, IEEE*, Jiashuo Jiang, *Member, IEEE*, Lei Deng, *Member, IEEE*, and Danny H.K. Tsang, *Life Fellow, IEEE*

Abstract—With the proliferation of Internet of Things (IoT) devices, the demand for addressing complex optimization challenges has intensified. The Lyapunov Drift-Plus-Penalty algorithm is a widely adopted approach for ensuring queue stability, and some research has preliminarily explored its integration with reinforcement learning (RL). In this paper, we investigate the adaptation of the Lyapunov Drift-Plus-Penalty algorithm for RL applications, deriving an effective method for combining Lyapunov Drift-Plus-Penalty with RL under a set of common and reasonable conditions through rigorous theoretical analysis. Unlike existing approaches that directly merge the two frameworks, our proposed algorithm, termed Lyapunov drift-plus-penalty method tailored for reinforcement learning with queue stability (LDPTRLQ) algorithm, offers theoretical superiority by effectively balancing the greedy optimization of Lyapunov Drift-Plus-Penalty with the long-term perspective of RL. Simulation results for multiple problems demonstrate that LDPTRLQ outperforms the baseline methods using the Lyapunov drift-plus-penalty method and RL, corroborating the validity of our theoretical derivations. The results also demonstrate that our proposed algorithm outperforms other benchmarks in terms of compatibility and stability.

Index Terms—Online Optimization, Reinforcement Learning, Lyapunov Drift-Plus-Penalty

I. INTRODUCTION

The Lyapunov drift-plus-penalty framework has emerged as a widely adopted technique across diverse domains, including backpressure routing [1], Internet of Things (IoT) devices, adaptive video streaming [2], and edge computing [3]. This method excels in stabilizing queues—both physical and virtual—within queuing networks and other stochastic systems while simultaneously optimizing time-averaged performance objectives, such as minimizing a penalty function [4]. Its major strengths are implementation efficiency and simplicity. Unlike traditional approaches, the Lyapunov drift-plus-penalty method requires no prior knowledge of underlying statistical distributions (e.g., probability density functions of

random channel conditions). Instead, it relies solely on greedy optimization of arbitrary utility functions at each time step. This lightweight and scalable design eliminates the need for complex probabilistic modeling, rendering the technique highly adaptable and broadly applicable [5].

Although the Lyapunov drift-plus-penalty method offers simplicity in implementation [6], its effectiveness is often constrained by the complexity of penalty functions inherent in many optimization problems. These penalty functions, typically dictated by the optimization objectives of specific scenarios, are frequently complex and non-convex. While they remain computable, sometimes they do not inherently possess greedy properties, limiting the method's applicability in such cases [7]. Consequently, researchers have explored alternative approaches to address these challenges.

Recent advancements in data-driven reinforcement learning (RL) present a promising alternative for tackling online optimization problems in queuing systems [8]. The RL framework adopts a model-free approach, leveraging deep neural networks to directly learn satisfactory mappings from states (e.g., time-varying system parameters) to actions (e.g., decision-making and resource allocation strategies for specific applications) [9]. Through iterative interactions with the environment, RL maximizes a reward (e.g., the optimization objective), effectively bypassing the need for detailed analysis of complex, non-convex optimization landscapes. This method autonomously learns from past experiences without requiring manually labeled training data, making it particularly well-suited for queuing problems that demand real-time, online implementation.

Recent research has predominantly explored direct combinations of the Lyapunov drift-plus-penalty method and RL algorithms to tackle optimization challenges in stochastic systems [10]. However, this integration poses notable difficulties. The Lyapunov drift-plus-penalty method operates as a greedy algorithm, optimizing decisions based on immediate outcomes, whereas RL employs non-greedy exploration to maximize long-term rewards. Directly employing the Lyapunov optimization objective as the reward function in RL fails to account for this inherent incompatibility, raising concerns about the efficacy and robustness of such approaches. Furthermore, devising a method that effectively unifies these two frameworks remains a complex challenge, as their underlying principles diverge significantly.

Wenhan Xu is with the Internet of Things Thrust, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong 511400, China (e-mail: wxube@connect.ust.hk).

Jiashuo Jiang is with Department of Industrial Engineering and Decision Analytics, Hong Kong University of Science and Technology, Hong Kong 99907, China (e-mail: jsjiang@ust.hk).

Lei Deng is with the Internet of Things Thrust, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong 511400, China (e-mail: leideng@hkust-gz.edu.cn).

Danny Hin-Kwok Tsang is with the Internet of Things Thrust, Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong 511400, China, and also with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Clear Water Bay, Hong Kong SAR, China (e-mail: eetsang@ust.hk).

A. Related Work

The Lyapunov drift-plus-penalty method, first introduced by Neely in 2010 [11] to stabilize queues while optimizing specific objectives, has since found widespread application in queuing-related optimization problems, including backpressure routing [1], IoT resource allocation [12], and edge computing [13]. This approach transforms long-term queue stability goals into per-slot minimization of a drift-plus-penalty term, employing a greedy optimization strategy that requires no prior statistical knowledge of the system (e.g., stochastic probability density functions). Bracciale et al. [14] developed a system based on this method, enforcing not only average queue stability but also strict per-slot maximum queue (or delay) capacity constraints. Similarly, [15] proposed an online Lyapunov-based approach, investigating centralized and distributed schemes for joint task offloading and resource allocation in energy-constrained mobile edge computing networks. Dai et al. [16] addressed unmanned aerial vehicle (UAV)-assisted vehicular task offloading in vehicular edge computing networks, leveraging the drift-plus-penalty technique to convert long-term energy constraints into tractable real-time constraints. Furthermore, [17] implemented the Lyapunov drift-plus-penalty framework to obtain a dynamic resource allocation policy in a stochastic network optimization problem. [18] tackled task offloading and resource allocation for 5G-connected vehicles in mobile edge computing (MEC), constructing a Lyapunov function within the drift-plus-penalty framework to transform optimization objectives into per-slot offloading decisions that bound the objective's upper limit.

Concurrently, RL has emerged as a robust framework for data-driven optimization in dynamic environments. Model-free RL algorithms, typically leveraging deep neural networks, learn effective policies through iterative trial-and-error interactions, rendering them well-suited for a variety of online optimization problems [19]. In contrast to the Lyapunov drift-plus-penalty technique, which prioritizes per-slot optimization, RL focuses on maximizing long-term rewards, offering a fundamentally different approach to system dynamics. Implementing an RL framework requires only a standardized specification of actions, states, rewards, and state transitions [20]. This versatility has led to its widespread adoption across diverse real-world applications, including large language models [21], image recognition [22], and perfect-information games [23].

Recent efforts have sought to integrate the Lyapunov drift-plus-penalty method with RL to harness their complementary strengths [9], [24]–[27]. For instance, researchers in [24] incorporated Lyapunov optimization objectives directly into RL reward functions to address edge computing challenges in Digital Twin Networks. Similarly, [9] applied a hybrid framework to mobile edge computing, using Lyapunov terms to guide RL policy updates. In [25], a channel selection framework was proposed, combining machine learning, Lyapunov optimization, and matching theory to support industrial IoT implementations. Zhou et al. [26] proposed a predictable radio resource scheduling scheme leveraging Lyapunov-guided reinforcement learning with proximal policy optimization.

Their approach maximized the transmission rate of real-time services while ensuring jitter-bounded deterministic delay constraints for time-sensitive applications. Similarly, [9] proposed a Lyapunov optimization-based multi-agent Proximal Policy Optimization algorithm to solve the MEC problems for internet of vehicles systems. While these studies demonstrate performance improvements in specific contexts, they overlook a fundamental mismatch: the greedy nature of the Lyapunov drift-plus-penalty method conflicts with RL's focus on maximizing long-term rewards. This incompatibility often results in suboptimal policies or instability, particularly in queuing systems where non-convex complex objectives predominate [28]. Furthermore, redesigning the Lyapunov framework to align with RL's non-greedy dynamics remains largely unaddressed, leaving a research gap in tackling complex, real-time optimization challenges.

B. Motivation and Contribution

The rapid proliferation of portable IoT devices and applications has significantly increased the demand for optimization frameworks that ensure queue stability while addressing increasingly complex objectives. These scenarios, spanning domains such as backpressure routing and edge computing, often involve dynamic, non-convex optimization targets tied to practical performance metrics. Concurrently, advancements in artificial intelligence have prompted researchers to explore superior algorithmic alternatives, even in well-established fields like backpressure routing. The Lyapunov drift-plus-penalty method has emerged as a popular approach for optimizing application-specific objectives under queue stability constraints, leveraging its greedy, per-slot minimization strategy [11], [14]–[16], [18]. However, the dynamic, non-convex, and intricate nature of modern optimization goals has driven efforts to integrate this method with data-driven, model-free RL algorithms [9], [24]–[27]. While such hybrid approaches have demonstrated performance gains in specific contexts, a critical mismatch persists: the greedy optimization inherent to the Lyapunov framework conflicts with RL's focus on maximizing long-term rewards, often leading to suboptimal or unstable outcomes. Our prior research investigated joint energy consumption optimization and queue stability in intelligent reflecting surface-assisted fog computing [28], highlighting the potential of tailored approaches in complex scenarios.

Motivated by the insights from the aforementioned literature review, we propose a novel reformulation of the Lyapunov drift-plus-penalty method, adapting it to align with RL's non-greedy paradigm. This paper presents several key contributions to address the integration of the Lyapunov drift-plus-penalty method with RL for queuing system optimization:

- We conduct a theoretical analysis of the Lyapunov drift-plus-penalty method, employing deductive reasoning to derive a mathematical formulation tailored for RL environments. Through this analysis, we identify the underlying reasons why directly adopting the optimization objectives of the Lyapunov method as RL reward functions is suboptimal, revealing a mismatch between its greedy nature and RL's long-term reward optimization.

- Grounded in the practical significance of optimization objectives, we propose a set of basic theorems that such objectives should exhibit. Leveraging these theorems, we propose a Lyapunov drift-plus-penalty method tailored for reinforcement learning with queue stability (LDPTRLQ) algorithm, ensuring compatibility with its non-greedy dynamics while preserving queue stability.
- We validate the performance of the proposed RL-adapted Lyapunov drift-plus-penalty algorithm in **a mobile edge computing problem**, and compare it against the widely adopted original Lyapunov drift-plus-penalty-assisted RL approach. Numerical results demonstrate that our algorithm achieves better optimization performance and faster convergence. Additionally, comparisons with other state-of-the-art RL-based optimization methods in this domain highlight our approach's pronounced advantage in maintaining queue stability.
- We validate the performance of the proposed RL-adapted Lyapunov drift-plus-penalty algorithm in a routing problem. The results demonstrate that LDPTLQ achieves performance comparable to the established backpressure routing algorithm while surpassing other hybrid approaches that integrate Lyapunov Drift-Plus-Penalty with RL. Furthermore, LDPTLQ exhibits robust adaptability to a diverse set of complex optimization objectives, highlighting its versatility and effectiveness in dynamic network environments.
- Across a variety of queueing optimization problems, our proposed LDPTLQ algorithm demonstrates superior performance. These results underscore the algorithm's robust generality and scalability, highlighting its potential for broad applicability in diverse optimization scenarios.

C. Notations

Scalars and vectors are denoted by letters and bold letters, respectively. $|x|$ is the absolute value of the scalar x . $\|\mathbf{x}\|$ is the L_2 -norm of vector $\mathbf{x}$. $\mathbf{X}^T$, $\mathbf{X}^H$, and $|\mathbf{X}|$ are the transpose, conjugate transpose, and determinant norm of a matrix $\mathbf{X}$, respectively. The primary notations in this paper are summarized in TABLE. I.

II. SYSTEM MODEL AND BACKGROUND

In this section, we present the theoretical foundations of the Lyapunov drift-plus-penalty method as applied to queueing systems. This framework provides a structured approach to stabilize queues while optimizing performance objectives, laying the groundwork for our subsequent analysis and algorithm design.

A. Optimization Problem with Queueing System

Consider a system comprising N distinct queues. The queue lengths over time are represented by the vector:

$$\mathbf{Q}(t) = [Q_1(t), Q_2(t), \dots, Q_n(t), \dots, Q_N(t)], \quad (1)$$

where $Q_n(t) \geq 0$ denotes the length of the n^{th} queue at time slot $t \in \{0, 1, 2, 3, \dots\}$. To characterize the evolution of queue

TABLE I
PARAMETERS NOTATION

Parameters	Notation
n	n^{th} queue
N	Number of queues
Q	Length of queue
$\mathbb{E}\{x\}$	Expectation of x
$\mathbb{R}$	Real number field
d^{arr}	Arrival of the queue
d^{dep}	Departure of the queue
t	t^{th} time slot
ΔT	Length of time slot
C	Lyapunov drift-plus-penalty function
V	Weight between Lyapunov drift and penalty
$\bar{p}$	Time-averaged Lyapunov penalty
$p(t)$	Lyapunov penalty at time slot t
$\mathbf{a}$	Action of RL algorithm
$\mathbf{s}$	State of RL algorithm
$\mathcal{A}$	Set of feasible actions
$\mathcal{S}$	Set of feasible states
$\mathcal{R}$	Reward of RL
λ	Task arriving rate
K	Number of mobile users

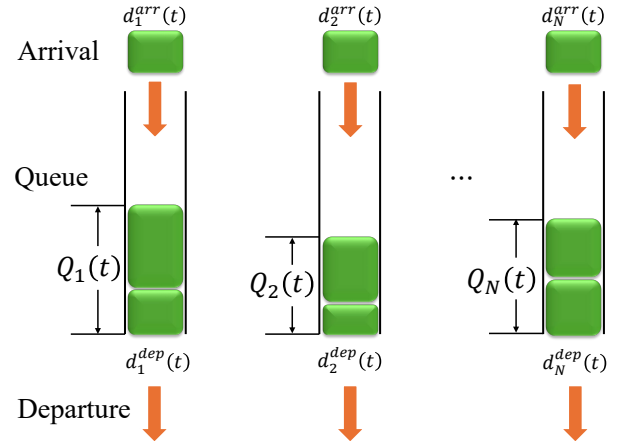


Fig. 1. System model of N distinct queues.

lengths, we denote the arrival process for the n^{th} queue at time slot t as $d_n^{arr}(t)$, and the departure process as $d_n^{dep}(t)$. The evolution of the queue lengths can be described by the following queueing model:

$$Q_n(t+1) = \max\{Q_n(t) - d_n^{dep}(t), 0\} + d_n^{arr}(t). \quad (2)$$

Definition 1. [6] A discrete time queue Q_n is mean rate stable if

$$\lim_{t \rightarrow \infty} \frac{\mathbb{E}\{Q_n(t)\}}{t} = 0, \quad (3)$$

where $\mathbb{E}\{Q_n(t)\}$ is the expectation of $Q_n(t)$.

Having established the queue dynamics, we now formulate the optimization problem central to the Lyapunov drift-plus-

penalty framework [14]. The objective is to find a policy to minimize the average penalty with the mean rate stable queue:

$$\begin{aligned} \mathcal{P}1 : \min \bar{p} \\ \text{s.t. } Q_n \text{ is mean rate stable, } \forall n, \end{aligned} \quad (4)$$

where $\bar{p}$ is the time-averaged penalty, which excludes queue stability constraints:

$$\bar{p} = \lim_{\tau \rightarrow \infty} \frac{\sum_{t=0}^{\tau-1} \mathbb{E}\{p(t)\}}{\tau}, \quad (5)$$

where $p(t)$ denotes the penalty incurred at time slot t . Different specific problems have different $p(t)$ expressions. In practical applications, $p(t)$ typically represents the time-averaged energy consumption or other cost-related metrics.

B. Lyapunov Drift-Plus-Penalty Framework

Having established the basic optimization problem, we now turn to the Lyapunov drift-plus-penalty framework. Central to this approach is the introduction of a Lyapunov function based on the queue vector $Q(t)$, defined as follows

$$LY(Q(t)) \triangleq \frac{1}{2} \sum_{n=1}^N Q_n^2(t), \quad (6)$$

where $LY(Q(t))$ is the Lyapunov function of $Q(t)$. Accordingly, the Lyapunov drift at time slot t is defined as the difference between the Lyapunov function values of consecutive time slots, expressed as

$$\begin{aligned} \Delta Q(t) &\triangleq \mathbb{E}\{LY(Q(t+1)) - LY(Q(t)) | Q(t)\} \\ &= \mathbb{E}\left\{\frac{1}{2} \sum_{n=1}^N Q_n^2(t+1) - \frac{1}{2} \sum_{n=1}^N Q_n^2(t) \middle| Q(t)\right\} \\ &= \mathbb{E}\left\{\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) \middle| Q(t)\right\}. \end{aligned} \quad (7)$$

Therefore, the Lyapunov drift-plus-penalty function at time slot t is

$$\begin{aligned} \mathcal{C}(t) &= \underbrace{\Delta Q(t)}_{\text{Drift}} + \underbrace{V\mathbb{E}\{p(t) | Q(t)\}}_{\text{Penalty}} \\ &= \underbrace{\mathbb{E}\left\{\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) \middle| Q(t)\right\}}_{\text{Drift}} \\ &\quad + \underbrace{V\mathbb{E}\{p(t) | Q(t)\}}_{\text{Penalty}}, \end{aligned} \quad (8)$$

where V is the weight between Lyapunov drift and penalty. By manipulating the controllable components of the optimization environment, the Lyapunov drift-plus-penalty method performs optimization at each time slot:

$$\begin{aligned} \mathcal{P}2 : \min_{\alpha(t)} \mathcal{C}(t) \\ \text{s.t. } \alpha(t) \in \mathcal{A}(t), \end{aligned} \quad (9)$$

where $\alpha(t)$ represents the action at time slot t that the optimizer needs to decide, selected from the set of feasible actions $\mathcal{A}(t)$, and the Lyapunov drift-plus-penalty function

$\mathcal{C}(t)$ is a function of the action $\alpha(t)$. Since the optimization process is confined to a single time slot and $Q(t)$ is a known quantity during per-slot optimization, the expectation terms in the Lyapunov drift-plus-penalty function $\mathcal{C}(t)$ can be replaced with their current values, yielding

$$\begin{aligned} \mathcal{P}3 : \min_{\alpha(t)} \frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) + Vp(t) \\ \text{s.t. } \alpha(t) \in \mathcal{A}(t), \end{aligned} \quad (10)$$

In several related works [9], [14], the arrival length $d^{\text{arr}}(t)$ and departure length $d^{\text{dep}}(t)$ of queues within a single time slot are treated as bounded quantities, significantly smaller than the queue length $Q(t)$. Consequently, these papers employ simplification and approximation techniques to transform problem $\mathcal{P}3$ into problem $\mathcal{P}4$ as follows

$$\begin{aligned} \mathcal{P}4 : \min_{\alpha(t)} \sum_{n=1}^N Q_n(t) (d_n^{\text{arr}}(t) - d_n^{\text{dep}}(t)) + Vp(t) \\ \text{s.t. } \alpha(t) \in \mathcal{A}(t), \end{aligned} \quad (11)$$

III. LYAPUNOV DRIFT-PLUS-PENALTY METHOD FOR REINFORCEMENT LEARNING

Typically, the greedy optimization problems $\mathcal{P}3$ and $\mathcal{P}4$ can be solved directly. However, when $p(t)$ exhibits a complex, non-convex form, solving $\mathcal{P}3$ and $\mathcal{P}4$ may become NP-hard, rendering them computationally intractable and resource-intensive. Furthermore, in practical optimization scenarios, the greedy minimization of the penalty $p(t)$ often fails to reduce its long-term time-averaged value, indicating weak greedy properties of the penalty term. In such cases, even if $\mathcal{P}3$ and $\mathcal{P}4$ are solvable, the resulting solutions yield suboptimal performance. This limitation has driven researchers to explore model-free optimization techniques, such as deep learning. In this section, we aim to derive a theoretically sound adaptation of the Lyapunov drift-plus-penalty method for RL applications, addressing these challenges through rigorous analysis.

A. Theoretical Derivation of the Proposed LDPTRLQ Method

1) *Derivation of the form of optimization problem:* The following theorem builds on the widespread application of the Lyapunov drift-plus-penalty method to queue stability problems. This method's established efficacy in such contexts provides the foundation for our subsequent adaptation to RL frameworks.

Theorem 1. *The Lyapunov drift-plus-penalty method provides an effective framework for addressing queue stability problems. Specifically, its effectiveness means that solving the optimization problem $\mathcal{P}3$ at each time slot can achieve the desired optimization objective while ensuring queue stability.*

Theorem 1 is proven in [11], and the successful application of the Lyapunov drift-plus-penalty method in [14]–[16], [18] further validates it. In RL methodologies, the optimization objective is typically transformed directly into the reward function, with the action space and state space derived from the original optimization problem's decision variables and feasible

domain, respectively. A key advantage of RL is that it requires only the ability to compute the reward—equivalent to the optimization objective—rendering approximation or simplification of the reward unnecessary. Such simplifications, as applied in transforming $\mathcal{P}3$ to $\mathcal{P}4$, can even degrade RL efficiency. Consequently, we focus solely on the original problem $\mathcal{P}3$. Building on Theorem 1, we deduce that the optimization problem we seek is equivalent to $\mathcal{P}3$. In terms of the equivalent optimization problem, we have Theorem 2.

Theorem 2. All problems equivalent to a given optimization problem can be characterized by applying a monotonically increasing function to its objective function:

$$\min_{\alpha \in \mathcal{A}} f(\alpha) \iff \min_{\alpha \in \mathcal{A}} \phi(f(\alpha)) \quad (12)$$

where ϕ is a monotonically increasing function with a single independent variable.

Proof. We have an optimization problem $\mathcal{P}3a$ defined as

$$\mathcal{P}3a : \min_{\alpha \in \mathcal{A}} f(\alpha), \quad (13)$$

and another optimization problem $\mathcal{P}3b$ defined as

$$\mathcal{P}3b : \min_{\alpha \in \mathcal{A}} g(\alpha). \quad (14)$$

where $g(\alpha) = \phi(f(\alpha))$, and $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is a monotonically increasing function, i.e., for any $x_1, x_2 \in \mathbb{R}$, if $x_1 < x_2$, then $\phi(x_1) < \phi(x_2)$.

To prove equivalence, we need to show that the optimal solutions of $\mathcal{P}3a$ and $\mathcal{P}3b$ are identical. Let $\alpha^* \in \mathcal{A}$ be an optimal solution to $\mathcal{P}$, so that

$$f(\alpha^*) \leq f(\alpha), \quad \forall \alpha \in \mathcal{A}. \quad (15)$$

Since ϕ is monotonically increasing, applying ϕ to both sides of the inequality preserves the order:

$$\phi(f(\alpha^*)) \leq \phi(f(\alpha)), \quad \forall \alpha \in \mathcal{A}. \quad (16)$$

Thus,

$$g(\alpha^*) = \phi(f(\alpha^*)) \leq \phi(f(\alpha)) = g(\alpha), \quad \forall \alpha \in \mathcal{A}. \quad (17)$$

This implies that α^* is also an optimal solution to $\mathcal{Q}$.

Conversely, suppose $\beta^* \in \mathcal{X}$ is an optimal solution to $\mathcal{P}3b$, so that

$$g(\beta^*) = \phi(f(\beta^*)) \leq \phi(f(\alpha)) = g(\alpha), \quad \forall \alpha \in \mathcal{A}. \quad (18)$$

Since ϕ is monotonically increasing, it is injective (one-to-one). Thus, $\phi(f(\beta^*)) \leq \phi(f(\alpha))$ implies $f(\beta^*) \leq f(\alpha)$ for all $\alpha \in \mathcal{A}$, because if $f(\beta^*) > f(\alpha)$, then $\phi(f(\beta^*)) > \phi(f(\alpha))$, contradicting the optimality of β^* in $\mathcal{P}3b$. Hence, β^* is also an optimal solution to $\mathcal{P}3a$.

Therefore, the optimal solution sets of $\mathcal{P}3a$ and $\mathcal{P}3b$ are identical, proving that $\mathcal{P}3b$, defined by $g(\alpha) = \phi(f(\alpha))$ with ϕ monotonically increasing, is equivalent to $\mathcal{P}3a$. $\square$

Indeed, all optimization problems equivalent to $\mathcal{P}3$ with different objectives can be expressed in the following form

$$\mathcal{P}5 : \min_{\alpha(t)} f_{1,t} \left(\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) + Vp(t) \right) \quad (19)$$

monotonically increasing form of $\mathcal{P}3$

s.t. $\alpha(t) \in \mathcal{A}(t)$,

where $f_{1,t}, \forall t$ is a monotonically increasing function with a single independent variable. Since $f_{1,t}(x), \forall t$ achieves its minimum value when x attains the minimum within its domain, this transformation preserves the solution to the minimization problem.

2) Derivation of undetermined term in the optimization problem: The reward function for the RL algorithm we seek for $\mathcal{P}5$ can be formulated as follows

$$\mathcal{R}(t) = -f_{1,t} \left(\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) + Vp(t) \right). \quad (20)$$

Since our goal is to develop a method applicable to arbitrary scenarios, we consider a special case of the optimization problem: one where the queue length remains zero at all times and the greedy minimization of the original objective achieves the minimization of its long-term time-averaged value. We have the original problem $\mathcal{P}6$ and the target problem $\mathcal{P}7$ suitable for RL:

$$\mathcal{P}6 : \min_{\alpha(t)} Vp(t) \quad \begin{array}{l} \text{minimization of original objective} \\ \text{original problem} \end{array} \quad (21)$$

s.t. $\alpha(t) \in \mathcal{A}(t)$,

and

$$\mathcal{P}7 : \min_{\alpha(t)} f_{1,t}(Vp(t)) \quad \begin{array}{l} \text{queue length remains zero at all} \\ \text{time} \\ \text{suitable for RL} \end{array} \quad (22)$$

s.t. $\alpha(t) \in \mathcal{A}(t)$.

The greedy minimization of the original objective achieves the minimization of its long-term time-averaged value. To achieve the long-term time-averaged value of $\mathcal{P}6$, we need to solve

$$\mathcal{P}8 : \min_{\alpha} \lim_{\tau \rightarrow \infty} \frac{\sum_{t=0}^{\tau-1} Vp(t)}{\tau} \quad (23)$$

s.t. $\alpha \in \mathcal{A}$.

$\mathcal{P}8$ is equivalent to

$$\mathcal{P}9 : \min_{\alpha} \lim_{\tau \rightarrow \infty} \frac{f_{1,t'}(\sum_{t=0}^{\tau-1} Vp(t))}{\tau}, \forall t', \quad (24)$$

s.t. $\alpha \in \mathcal{A}$,

محاسبه این دردمس داره چون باید تا آخرین لحظه الگوریتم اجرا بشه از این رو این برای تئوری مناسب است

where t' can be any integer. The RL algorithm to solve $\mathcal{P}7$ will look for the maximum long-term time-averaged value of reward

$$\mathcal{P}10 : \max_{\alpha} \lim_{\tau \rightarrow \infty} - \frac{\sum_{t=0}^{\tau-1} f_{1,t}(Vp(t))}{\tau} \quad (25)$$

s.t. $\alpha \in \mathcal{A}$.

$\mathcal{P}10$ is equivalent to

$$\mathcal{P}11 : \min_{\alpha} \lim_{\tau \rightarrow \infty} \frac{\sum_{t=0}^{\tau-1} f_{1,t}(Vp(t))}{\tau} \quad (26)$$

s.t. $\alpha \in \mathcal{A}$.

محاسبه این در لحظه انجام میشه از این رو این برای الگوریتم و پیاده سازی

According to the assumption of this special case, $\mathcal{P}11$ is equivalent to $\mathcal{P}9$. Therefore,

$$f_{1,t'} \left(\sum_{t=0}^{\tau-1} Vp(t) \right) = f_2 \left(\sum_{t=0}^{\tau-1} f_{1,t}(Vp(t)) \right), \forall t', \quad (27)$$

where f_2 is a monotonically increasing function with a single independent variable, and Eq.(28) holds true for any value of $\{p(t), \tau, V\}$. Let $\tau = 1, t' = 0$, we can get

$$f_{1,0}(Vp(0)) = f_2(f_{1,0}(Vp(0))). \quad (28)$$

Let $x = f_{1,0}(Vp(0))$ and we can get $f_2(x) = x$. Therefore,

$$f_{1,t'}\left(\sum_{t=0}^{\tau-1} Vp(t)\right) = \sum_{t=0}^{\tau-1} f_{1,t}(Vp(t)), \forall t'. \quad (29)$$

Lemma 1. *Eq.(29) holds true for any value of $\{p(t), \tau, V, t'\}$ if and only if $f_{1,t}(x) = a_1x + a_1a_{0,t}$ with $a_1 > 0$.*

We are indifferent to whether the Lyapunov drift-plus-penalty function is scaled to $\mathcal{C}a_1$ or $\mathcal{C}/a_1$. Accordingly, for simplicity and without loss of generality, we set $a_1 = 1$. Therefore, we get

$$f_{1,t}(x) = x + a_{0,t}. \quad (30)$$

Based on the theoretical derivations presented above, the **reward** function suitable for RL, derived from the Lyapunov drift-plus-penalty method, is formulated as follows

$$\mathcal{R}(t) = -\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) - Vp(t) - \overset{\text{Maximum}}{a_{0,t}}. \quad (31)$$

The task is simplified to determining $a_{0,t}$. **Within a single time slot, $a_{0,t}$ is a constant and, thus, independent of any quantities that may vary within the slot. Consequently, $a_{0,t}$ depends solely on fixed values within the time slot. In this study, we investigate arbitrary problems satisfying the given expression, where the only common fixed value across time slots is the queue state information, denoted as $Q(t)$.** Therefore, we define $a_{0,t} = f_3(Q(t))$ as

$$\mathcal{R}(t) = -\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) - Vp(t) - f_3(Q(t)). \quad (32)$$

The objective is to optimize the long-term stability of the queue length; hence, the specific time at which a particular queue length occurs is of no concern. This leads to the following corollary

Lemma 2. *If the queue lengths at the time slot t and at the time slot $t+1$ are swapped while keeping the penalty term unchanged, the reward for reinforcement learning should remain equivalent.*

From Lemma 2, we have

$$\mathcal{R}(t) = \mathcal{R}^*(t), \quad (33)$$

where $\mathcal{R}^*(t)$ is the reward when the queue lengths at the time slot t and at the time slot $t+1$ are swapped:

$$\begin{aligned} \mathcal{R}^*(t) = & -\frac{1}{2} \sum_{n=1}^N (Q_n^2(t) - Q_n^2(t+1)) - Vp(t) \\ & - f_3(Q(t+1)). \end{aligned} \quad (34)$$

Therefore,

$$\begin{aligned} & -\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) - Vp(t) - f_3(Q(t)) \\ = & -\frac{1}{2} \sum_{n=1}^N (Q_n^2(t) - Q_n^2(t+1)) - Vp(t) - f_3(Q(t+1)). \end{aligned} \quad (35)$$

After moving the terms related to t to the right and $t+1$ to the left in Eq.(35) and simplifying, we can get

$$f_3(Q(t+1)) - \sum_{n=1}^N Q_n^2(t+1) = f_3(Q(t)) - \sum_{n=1}^N Q_n^2(t). \quad (36)$$

We can deduce that $f_3(Q(t)) - \sum_{n=1}^N Q_n^2(t)$ is a constant, and we let $f_3(Q(t)) - \sum_{n=1}^N Q_n^2(t) = a_0$. Therefore,

$$f_3(Q(t)) = \sum_{n=1}^N Q_n^2(t) + a_0. \quad (37)$$

B. Theoretical Derivation Conclusion

The constant term in the reward function for RL is not significant. Therefore, setting $a_0 = 0$ simplifies the expression without loss of generality. Hence,

$$\begin{aligned} \mathcal{R}(t) = & -\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) - Vp(t) - f_3(Q(t)) \\ = & -\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) - Q_n^2(t)) - Vp(t) \\ & - \sum_{n=1}^N Q_n^2(t) - a_0 \\ = & -\frac{1}{2} \sum_{n=1}^N (Q_n^2(t+1) + Q_n^2(t)) - Vp(t). \end{aligned} \quad (38)$$

Furthermore, in the context of reinforcement learning, the action is defined as $\alpha \in \mathcal{A}$, while the state $s \in \mathcal{S}$ encompasses system information, including the queue state. **Additionally, it is observed that when the proposed reinforcement learning algorithm is reduced to performing greedy optimization within a single time slot, it becomes equivalent to the original Lyapunov drift-plus-penalty method.** This equivalence validates the rationality of the proposed approach.

IV. NUMERICAL RESULTS AND ANALYSIS

We introduce several specific queueing-related practical problems to evaluate the performance of our proposed algorithm.

A. Mobile Edge Computing

We consider a system model for mobile edge computing, comprising K mobile users and a single base station as shown in Fig.2. The base station is equipped with an edge computing server dedicated to processing computational tasks. Each mobile user generates computational tasks in each time slot with a certain probability, **where task arrivals follow a Poisson process with an arrival rate of λ . The size of each task is uniformly distributed over the interval $(0, d_{\max})$.** For each mobile user, a task queue $Q_k(t)$ is maintained, with a local computing speed (bits/s) denoted by $c_k^L(t)$ and an upper bound on the computing speed (bits/s) given by $c_{\max,k}^L(t)$.

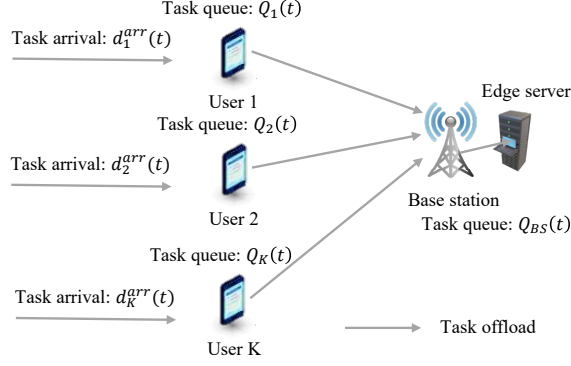


Fig. 2. System model of mobile edge computing.

The evolution of the task queue lengths at the k^{th} mobile user at time slot t can be described by the following queuing model:

$$Q_k(t+1) = \max\{Q_k(t) - d_k^{\text{dep}}(t), 0\} + d_k^{\text{arr}}(t), \forall k, \quad (39)$$

where $d_k^{\text{arr}}(t)$ is the total length of the newly generated task demand at time slot t , and $d_k^{\text{dep}}(t)$ represents the number of tasks departing from the task queue at time slot t , which comprises the tasks processed locally within the time slot t and those offloaded to the edge computing server:

$$d_k^{\text{dep}}(t) = R_k(t)\Delta T + c_k^L(t)\Delta T, \forall k, \quad (40)$$

where ΔT is the length of each time slot, and $R_k(t)$ is the offloading data rate (bit/s). Task offloading is facilitated through wireless communication. We adopt a fundamental wireless communication model wherein each user is assigned an independent orthogonal channel, each with a bandwidth of B . The transmission rate from user k to the base station, which corresponds to the task offloading rate, is given by

$$R_k(t) = B \log_2 \left(1 + \frac{\omega_k^{\text{WL}}(t)P_k(t)}{\sigma} \right), \forall k, \quad (41)$$

where $\omega_k^{\text{WL}}(t)$ denotes the channel gain at time slot t , $P_k(t)$ represents the transmission power from user k to the base station at time slot t , and σ is the channel noise power. The evolution of the task queue lengths at the edge server at the base station at time slot t can be described by the following queuing model

$$Q_{\text{BS}}(t+1) = \max\{Q_{\text{BS}}(t) - d_{\text{BS}}^{\text{dep}}(t), 0\} + d_{\text{BS}}^{\text{arr}}(t), \quad (42)$$

where $d_{\text{BS}}^{\text{arr}}(t)$ is the total length of the newly received task demand at the base station at time slot t :

$$d_{\text{BS}}^{\text{arr}}(t) = \sum_{k=1}^K R_k(t)\Delta T, \quad (43)$$

and $d_{\text{BS}}^{\text{dep}}(t)$ represents the number of tasks departing from the task queue $Q_{\text{BS}}(t)$ at time slot t , which comprises the tasks processed within the time slot t at the edge server:

$$d_{\text{BS}}^{\text{dep}}(t) = c_{\text{BS}}^E(t)\Delta T, \quad (44)$$

where $c_{\text{BS}}^E(t)$ is the task processing speed (bit/s) at the edge server at the base station at time slot t with an upper bound given by $c_{\text{max,BS}}^E(t)$. The energy consumption within each time slot comprises both the energy expended on communication and the energy expended on computation:

$$\begin{aligned} E^A(t) &= E^{\text{WC}}(t) + E^{\text{CC}}(t) \\ &= \sum_{k=1}^K P_k(t)\Delta T + \sum_{k=1}^K \eta_k c_k^L(t)\Delta T + \eta_{\text{BS}} c_{\text{BS}}^E(t)\Delta T, \end{aligned} \quad (45)$$

where η_k is the computing energy consumption per bit when the task is processed at the k^{th} mobile user, and η_{BS} is the computing energy consumption per bit when the task is processed at the edge server at the base station.

The reward function at time slot t of the proposed LDP-TRLQ algorithm, as derived in Eq. (35) of Section III, is applied to this specific edge computing problem, resulting in the following formula

$$\begin{aligned} \mathcal{R}(t) &= -\frac{1}{2} \sum_{k=1}^N (Q_k^2(t+1) + Q_k^2(t)) \\ &\quad - \frac{1}{2} (Q_{\text{BS}}^2(t+1) + Q_{\text{BS}}^2(t)) - V E^A(t). \end{aligned} \quad (46)$$

The action at time slot t encompasses the local computing speed of the mobile user, the computing speed of the edge computing server, and the power allocation for wireless communication:

$$\mathbf{a}(t) = \{c_k^L(t), c_{\text{BS}}^E(t), P_k(t), \forall k\}. \quad (47)$$

The state at time slot t comprises the task queue length at the mobile user, the task queue length at the edge computing server, and the channel gain of the wireless communication:

$$\mathbf{s}(t) = \{Q_k(t), Q_{\text{BS}}(t), \omega_k^{\text{WL}}(t), \forall k\}. \quad (48)$$

Our simulation system model consists of 1 base station with 1 edge computing server, serving 10 mobile users. The type of reinforcement learning agent is proximal policy optimization (PPO). The PPO agent has 5 fully connected layers with 64 neurons per layer equipped with the ReLU activation function. We set the PPO clipping parameter to 0.2. The discount factor is 0.95. The maximum number of steps per episode is 500, and the maximum number of episodes is 1000. The simulation environment is equipped with an Intel® Core™ i9-13900HX CPU and an RTX 4080 GPU. The simulation settings are also shown in Table II.

To verify the efficiency of our proposed algorithm, we performed a comparative analysis against several different algorithms. These comparisons aim to highlight the performance enhancements achieved through our method under various standard scenarios used in the field:

- **Original LDPRLQ:** This is the original Lyapunov Drift Plus Penalty with Reinforcement Learning (LDPRLQ) algorithm, as utilized in several academic papers [24]. It directly uses the negative of the Lyapunov drift plus penalty function as the reward for reinforcement learning.

TABLE II
SIMULATION SETTINGS

Parameter	Value	Parameter	Value
Bandwidth	10kHz	Noise	3.16×10^{-11}
$c_{\max,k}^L$	1000	$c_{\max,BS}^E$	5000
λ	2	K	10
Activ. func.	ReLU	PPO agent	5 layers
Layer	64 neurons	Mini batch	128
Reply memory	10000	Discount factor	0.95
Steps per episode	500	Maximum episode	1000

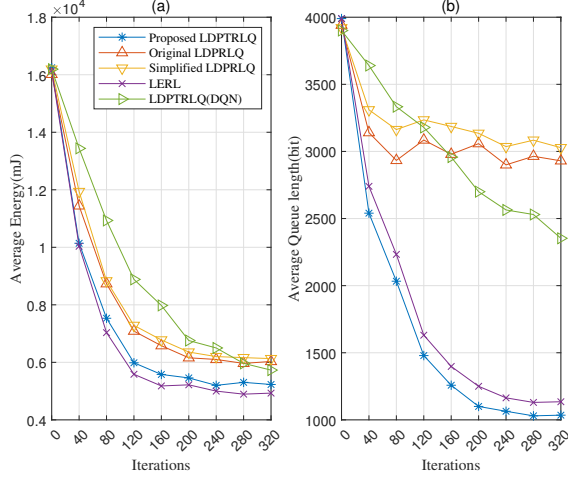


Fig. 3. The convergence behavior of our proposed LDPTLQ with benchmarks.

- Simplified LDPRLQ: A variant adopted in [9], where the Lyapunov drift plus penalty function is approximated as follows

$$\mathcal{C}(t) = \sum_{n=1}^N Q_n(t)(Q_n(t+1) - Q_n(t)) + Vp(t). \quad (49)$$

The negative of the objective is used as the reward for reinforcement learning.

- LERL: Latency-energy tradeoff with RL. An algorithm unrelated to the Lyapunov drift plus penalty framework, which optimizes a weighted sum of latency and energy consumption, as presented in [29]. In the context of our problem, latency corresponds to the queue length, thus directly optimizing the weighted sum of queue length and the penalty term.
- LDPTLQ(DQN): A modification of our proposed algorithm where the default PPO agent is replaced with a Deep Q-Network (DQN) agent.

1) *Convergence speed comparison:* Fig.3 illustrates the convergence behavior of the proposed LDPTLQ algorithm alongside four baseline methods. Since the trade-off coefficient in the LERL algorithm differs in meaning from the Lyapunov weight V in our proposed approach, a preliminary traversal of LERL's trade-off coefficient was conducted. This enabled the selection of a coefficient value that aligns the energy consumption and queue length metrics with those of our

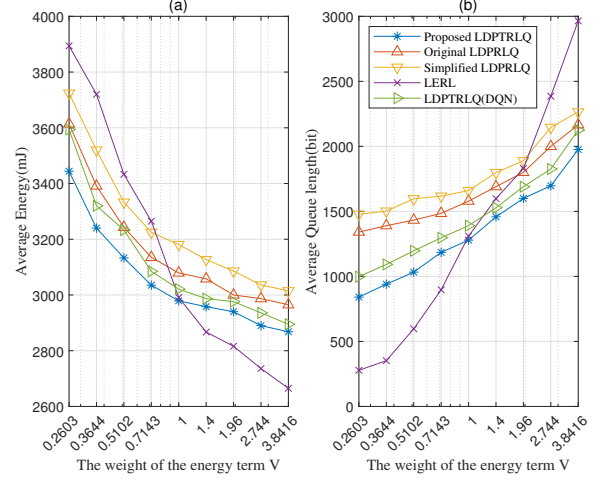


Fig. 4. The energy and queue length behavior of our proposed LDPTLQ with benchmarks under varying weights.

algorithm, facilitating a fair comparison under consistent conditions. Simulation results reveal that the proposed LDPTLQ algorithm achieves a balanced trade-off, significantly reducing both average energy consumption and average queue length. Its convergence speed surpasses that of Original LDPRLQ, Simplified LDPRLQ, and LDPTLQ(DQN), demonstrating enhanced optimization efficiency. While the LERL algorithm yields results comparable to our method, it requires careful tuning of the trade-off coefficient to ensure queue stability, whereas our algorithm inherently maintains stability without such adjustments. Furthermore, the slower convergence of LDPTLQ(DQN) compared to the proposed LDPTLQ suggests that, under these problem conditions, the PPO agent outperforms the DQN agent. The superior convergence speed of our LDPTLQ over Original LDPRLQ and Simplified LDPRLQ corroborates our theoretical derivations, underscoring deficiencies in these baseline methods that our approach effectively mitigates.

2) *Impact of different Lyapunov weight of proposed LDPTLQ and benchmarks:* Fig.4 presents the performance of the proposed LDPTLQ algorithm alongside four baseline methods under varying Lyapunov weight conditions. Given that the trade-off weight in the LERL algorithm differs in interpretation from the Lyapunov weight in our approach, a standardization of the weight factors was performed in advance. Specifically, a coarse traversal of the trade-off coefficient in LERL was conducted, and the weight yielding energy consumption and queue length values closest to those of the proposed LDPTLQ algorithm was normalized to 1. This normalization enables a consistent comparison across the algorithms.

Simulation results demonstrate that the proposed LDPTLQ algorithm outperforms Original LDPRLQ and Simplified LDPRLQ in terms of both average queue length and average energy consumption across all tested conditions. This improvement validates our theoretical findings, confirming that the proposed algorithm enhances the Lyapunov Drift-Plus-Penalty framework for reinforcement learning applications. In contrast,

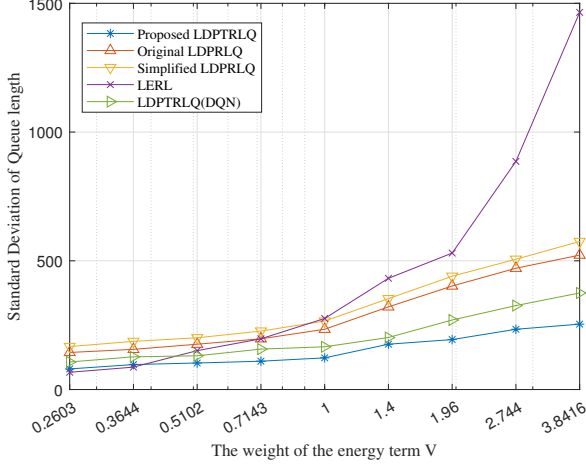


Fig. 5. The standard deviation queue length behavior of our proposed LDPTRLQ with benchmarks under varying weights.

LDPTRLQ(DQN) consistently exhibits inferior performance relative to the proposed LDPTRLQ, indicating that the PPO agent is more effective than the DQN agent in this application context. While the LERL algorithm achieves performance comparable to that of LDPTRLQ, it exhibits greater sensitivity to weight variations, making it challenging to identify a weight that ensures queue stability. These findings highlight the robustness and adaptability of the proposed LDPTRLQ algorithm.

Fig.5 illustrates the queue length standard deviation of the proposed LDPTRLQ algorithm and four baseline methods under varying Lyapunov weight conditions. The same weight factor standardization approach as described previously was applied. The results indicate that the proposed LDPTRLQ algorithm achieves the smallest queue length standard deviation in most scenarios, with LERL exhibiting a marginally lower standard deviation only when the queue weight factor is significantly elevated. This suggests that our proposed algorithm ensures superior queue stability compared to all baseline methods. The enhanced stability of LDPTRLQ stems from its incorporation of the queue length as a squared term in the reinforcement learning reward function, in contrast to LERL, which uses a linear term. Consequently, LERL is less sensitive to occasional excessive queue lengths, functioning primarily as a trade-off mechanism between queue length and energy consumption rather than an optimization strategy prioritizing queue stability. Similarly, the proposed LDPTRLQ algorithm consistently outperforms Original LDPRLQ and Simplified LDPRLQ in terms of queue length standard deviation, demonstrating an improvement over conventional approaches that integrate Lyapunov Drift-Plus-Penalty with reinforcement learning. Furthermore, the superior performance of LDPTRLQ over LDPTRLQ(DQN) reinforces the finding that, in this application context, the PPO agent outperforms the DQN agent.

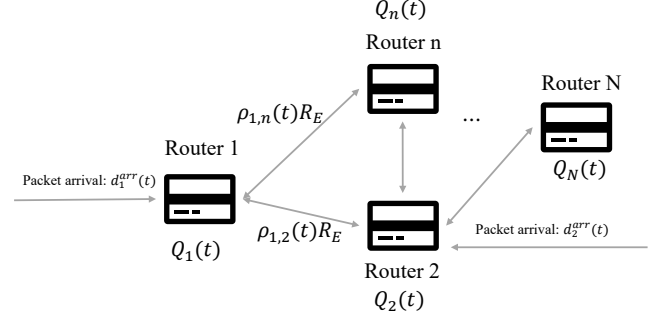


Fig. 6. System model of network routing.

B. Network Routing

In our simulation, we developed a method for constructing a randomly generated network topology for analysis as shown in Fig.6. The generation process begins with the creation of 20 nodes, each assigned a random two-dimensional coordinate (x, y) . The values for x and y are uniformly distributed between 0 and 100, ensuring that the node distribution covers the entire designated area evenly. Each node in this topology is then assessed for connectivity based on its proximity to other nodes. Specifically, each node is programmed to potentially connect to its 5 nearest neighbors. The probability of each of these connections being established is set at 50%, introducing a stochastic element to the network structure that mimics real-world irregularities and network dynamics. If the topology graphs are separated, we connect the closest nodes between them. The achievable data rate with each connection, denoted as R , is calculated using the formula $R = \rho R_E$, where ρ represents a scaling factor and R_E is the average data rate between the nodes. The scaling factor ρ is itself a variable, uniformly distributed between 0.3 and 1.7, which allows the connection speed to reflect a range of potential network conditions, from favorable to adverse.

1) *Impact of different arriving rate of proposed LDPTRLQ and benchmarks:* In [11], the Lyapunov drift theory (with $V = 0$, i.e., without any penalty term) was employed to mathematically analyze the conventional backpressure routing algorithm. These studies demonstrated that the backpressure routing algorithm is consistent with the Lyapunov drift framework. Specifically, under the condition $V = 0$, we can rewrite $\mathcal{P}4$ as

$$\begin{aligned} \mathcal{P}12 : \min_{\alpha(t)} & \sum_{n=1}^N Q_n(t)(d_n^{\text{arr}}(t) - d_n^{\text{dep}}(t)) \\ \text{s.t. } & \alpha(t) \in \mathcal{A}(t), \end{aligned} \quad (50)$$

where $\alpha(t)$ denotes the routing action, represented by a preconfigured set of service rate patterns. These patterns are designed such that each queue has exactly one next-hop routing target.

Problem $\mathcal{P}4$ is formulated as a linear programming problem, which can be efficiently solved using the CVX optimization toolbox. This solution approach is equivalent to the backpressure routing algorithm. Similarly, our LDPTRLQ proposed algorithm is capable of addressing this routing problem.

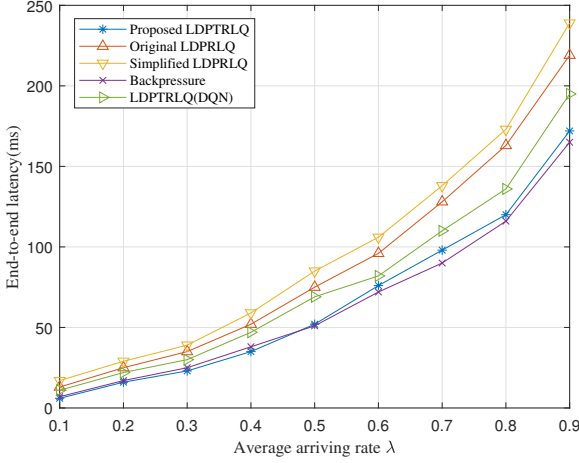


Fig. 7. The end-to-end latency versus the average arriving rate of our proposed LDPTLQ and benchmarks.

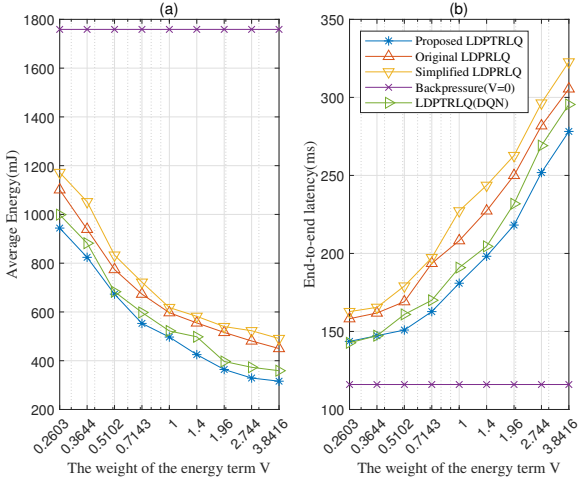


Fig. 8. The end-to-end latency versus the average arriving rate of our proposed LDPTLQ and benchmarks.

2) *Impact of different arriving rate of proposed LDPTLQ and benchmarks:* The performance of our proposed LDPTLQ algorithm, as illustrated in Fig.7, closely approximates that of the conventional backpressure routing algorithm. Notably, under low network congestion, LDPTLQ exhibits a slight performance advantage over the backpressure algorithm. However, despite this advantage under specific conditions, the computational resource consumption of our proposed algorithm significantly exceeds that of the backpressure algorithm, which requires minimal computational resources. Consequently, in the absence of a penalty term, the LDPTLQ algorithm does not outperform the backpressure algorithm. Furthermore, the higher latency observed with LDPTLQ(DQN) suggests that the PPO agent outperforms the DQN agent in this environment. Numerical results also demonstrate that our proposed LDPTLQ algorithm surpasses other algorithms combining Lyapunov drift-plus-penalty with reinforcement learning.

3) *Impact of different Lyapunov weight of proposed LDPTLQ and benchmarks:* We introduce the energy consumption model:

$$E(t) = \eta_2 R^2(t) + \eta_1 R(t), \quad (51)$$

where η_1 and η_2 are known coefficients, and $R(t)$ is the transmission data rate at time slot t . Furthermore, we subsequently address the comprehensive Lyapunov drift-plus-penalty optimization framework. Incorporating complex penalty terms into the backpressure routing algorithm is challenging due to the non-convex nature of the optimization problem, which renders it difficult to solve. Consequently, for the backpressure routing algorithm, we fix the trade-off parameter $V = 0$. Simulation results demonstrate that the proposed LDPTLQ algorithm outperforms all other algorithms except the backpressure routing algorithm, highlighting its superiority over existing approaches combining Lyapunov drift-plus-penalty with reinforcement learning. Moreover, while accounting for energy consumption, our algorithm achieves latency performance comparable to that of the backpressure algorithm to a certain extent. This ability to incorporate complex optimization factors while maintaining effective routing performance represents a key advantage of the proposed LDPTLQ algorithm over the backpressure routing algorithm.

V. CONCLUSION

In this paper, we proposed LDPTLQ, an enhanced algorithm integrating the Lyapunov Drift-Plus-Penalty framework with RL to address the optimization challenges posed by emerging complicated systems. Through theoretical derivations, we established an effective approach that outperforms conventional methods by effectively combining the short-term greediness of Lyapunov Drift-Plus-Penalty with the long-term optimization capabilities of RL. Simulation results in the MEC environment demonstrate that LDPTLQ consistently surpasses baseline algorithms. These findings validate our theoretical analysis, highlighting the algorithm's robustness and efficiency. While LERL exhibits competitive performance, its sensitivity to weight tuning underscores LDPTLQ's practical advantage in maintaining queue stability without extensive parameter adjustments. Simulation results in the routing environment demonstrate that, compared to the conventional backpressure routing algorithm, the proposed LDPTLQ algorithm effectively accommodates a wide range of complex optimization objectives. Furthermore, it outperforms other algorithms that integrate Lyapunov drift-plus-penalty with reinforcement learning. Future work will focus on validating these results in real-world IoT deployments and exploring adaptive weight mechanisms to further enhance performance under dynamic conditions.

REFERENCES

- [1] J. Núñez-Martínez and J. Mangues-Bafalluy, "Distributed Lyapunov drift-plus-penalty routing for wifi mesh networks with adaptive penalty weight," in *2012 IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*. IEEE, 2012, pp. 1–6.
- [2] N. Li, Y. Hu, Y. Chen, and B. Zeng, "Lyapunov optimized resource management for multiuser mobile video streaming," *IEEE Trans. Circuits Systems Video Technol.*, vol. 29, no. 6, pp. 1795–1805, 2018.

- [3] T. Shuminoski and T. Janevski, "Lyapunov optimization framework for 5G mobile nodes with multi-homing," *IEEE Commun. Letters*, vol. 20, no. 5, pp. 1026–1029, 2016.
- [4] H. Yu and M. J. Neely, "On the convergence time of the drift-plus-penalty algorithm for strongly convex programs," in *2015 54th IEEE Conference on Decision and Control (CDC)*. IEEE, 2015, pp. 2673–2679.
- [5] M. Hadi, M. R. Pakravan, and E. Agrell, "Dynamic resource allocation in metro elastic optical networks using Lyapunov drift optimization," *J. Optical Commun. Network.*, vol. 11, no. 6, pp. 250–259, 2019.
- [6] M. Neely, *Stochastic network optimization with application to communication and queueing systems*. Springer Nature, 2022.
- [7] S. Kerimov, I. Ashlagi, and I. Gurvich, "On the optimality of greedy policies in dynamic matching," *Operations Research*, vol. 73, no. 1, pp. 560–582, 2025.
- [8] J. Yu, X. Liu, Y. Gao, C. Zhang, and W. Zhang, "Deep learning for channel tracking in IRS-assisted UAV communication systems," *IEEE Trans. Wireless Commun.*, 2022.
- [9] S. Bi, L. Huang, H. Wang, and Y. J. A. Zhang, "Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 11, pp. 7519–7537, 2021.
- [10] X. Huang, S. Leng, S. Maharjan, and Y. Zhang, "Multi-agent deep reinforcement learning for computation offloading and interference coordination in small cell networks," *IEEE Trans. Vehicular Technol.*, vol. 70, no. 9, pp. 9282–9293, 2021.
- [11] M. Neely, *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers, 2010.
- [12] D. U. Kim, K. Kim, S. M. Kang, S.-B. Park, and C. S. Hong, "When Lyapunov drift meets DRL: Energy efficient resource allocation for IoT data collecting," in *2023 International Conference on Advanced Technologies for Communications (ATC)*. IEEE, 2023, pp. 382–387.
- [13] C. Xu, P. Zhang, and H. Yu, "Lyapunov-guided resource allocation and task scheduling for edge computing cognitive radio networks via deep reinforcement learning," *IEEE Sensors J.*, 2025.
- [14] L. Bracciale and P. Loreti, "Lyapunov drift-plus-penalty optimization for queues with finite capacity," *IEEE Commun. Lett.*, vol. 24, no. 11, pp. 2555–2558, 2020.
- [15] H. Jiang, X. Dai, Z. Xiao, and A. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," *IEEE Trans. Mobile Computing*, vol. 22, no. 7, pp. 4000–4015, 2022.
- [16] X. Dai, Z. Xiao, H. Jiang, and J. C. Lui, "UAV-assisted task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Computing*, vol. 23, no. 4, pp. 2520–2534, 2023.
- [17] C.-H. Hu, Z. Chen, and E. G. Larsson, "Energy-efficient federated edge learning with streaming data: A lyapunov optimization approach," *IEEE Trans. Commun.*, 2024.
- [18] G. Sun, Z. Wang, H. Su, H. Yu, B. Lei, and M. Guizani, "Profit maximization of independent task offloading in MEC-enabled 5G internet of vehicles," *IEEE Trans. Intelligent Transportat. Systems*, 2024.
- [19] Q. Meng, S. Hussain, F. Luo, Z. Wang, and X. Jin, "An online reinforcement learning-based energy management strategy for microgrids with centralized control," *IEEE Trans. Industry Applications*, 2024.
- [20] H. Wang, Z. Liu, G. Hu, X. Wang, and Z. Han, "Offline meta-reinforcement learning for active pantograph control in high-speed railways," *IEEE Trans. Industrial Informatics*, 2024.
- [21] Y. Cao, H. Zhao, Y. Cheng, T. Shu, Y. Chen, G. Liu, G. Liang, J. Zhao, J. Yan, and Y. Li, "Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods," *IEEE Trans. Neural Networks Learning Systems*, 2024.
- [22] J. Valente, J. António, C. Mora, and S. Jardim, "Developments in image processing using deep learning and reinforcement learning," *Journal of Imaging*, vol. 9, no. 10, p. 207, 2023.
- [23] T. Nakayashiki and T. Kaneko, "Maximum entropy reinforcement learning in two-player perfect information games," in *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2021, pp. 01–08.
- [24] Y. Dai, K. Zhang, S. Maharjan, and Y. Zhang, "Deep reinforcement learning for stochastic computation offloading in digital twin networks," *IEEE Trans. Industrial Informatics*, vol. 17, no. 7, pp. 4968–4977, 2020.
- [25] H. Liao, Z. Zhou, X. Zhao, L. Zhang, S. Mumtaz, A. Jolfaei, S. H. Ahmed, and A. K. Bashir, "Learning-based context-aware resource allocation for edge-computing-empowered industrial IoT," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4260–4277, 2019.
- [26] Y. Zhou, L. Feng, X. Jiang, W. Li, and F. Zhou, "Predictable wireless networked scheduling for bridging hybrid time-sensitive and real-time services," *IEEE Trans. Commun.*, vol. 72, no. 6, pp. 3664–3680, 2024.
- [27] Y. Jia, C. Zhang, Y. Huang, and W. Zhang, "Lyapunov optimization based mobile edge computing for internet of vehicles systems," *IEEE Trans. commun.*, vol. 70, no. 11, pp. 7418–7433, 2022.
- [28] W. Xu, Y. Yuan, and D. H. K. Tsang, "Enhancing fog computing through intelligent reflecting surface assistance: A Lyapunov driven reinforcement learning approach," in *2024 IEEE 10th World Forum on Internet of Things (WF-IoT)*. IEEE, 2024, pp. 759–764.
- [29] W. Xu, J. Yu, Y. Wu, and D. H. K. Tsang, "Energy-latency aware intelligent reflecting surface aided multi-cell mobile edge computing," *IEEE Trans. Green Commun. Network.*, vol. 8, no. 1, pp. 362–374, 2023.